

Hadoop与Spark

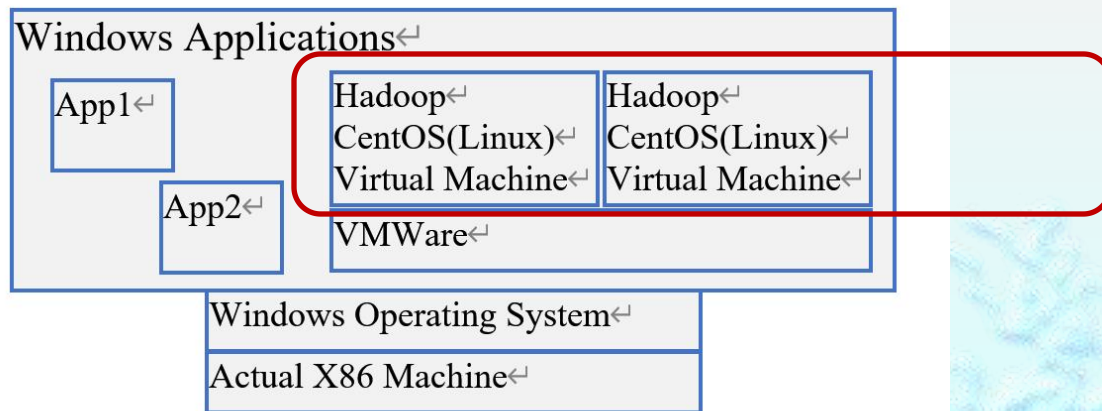
Hadoop安装与HDFS、MapReduce实验



Hadoop安装与HDFS、MapReduce实验

◆ 新建虚拟机集群

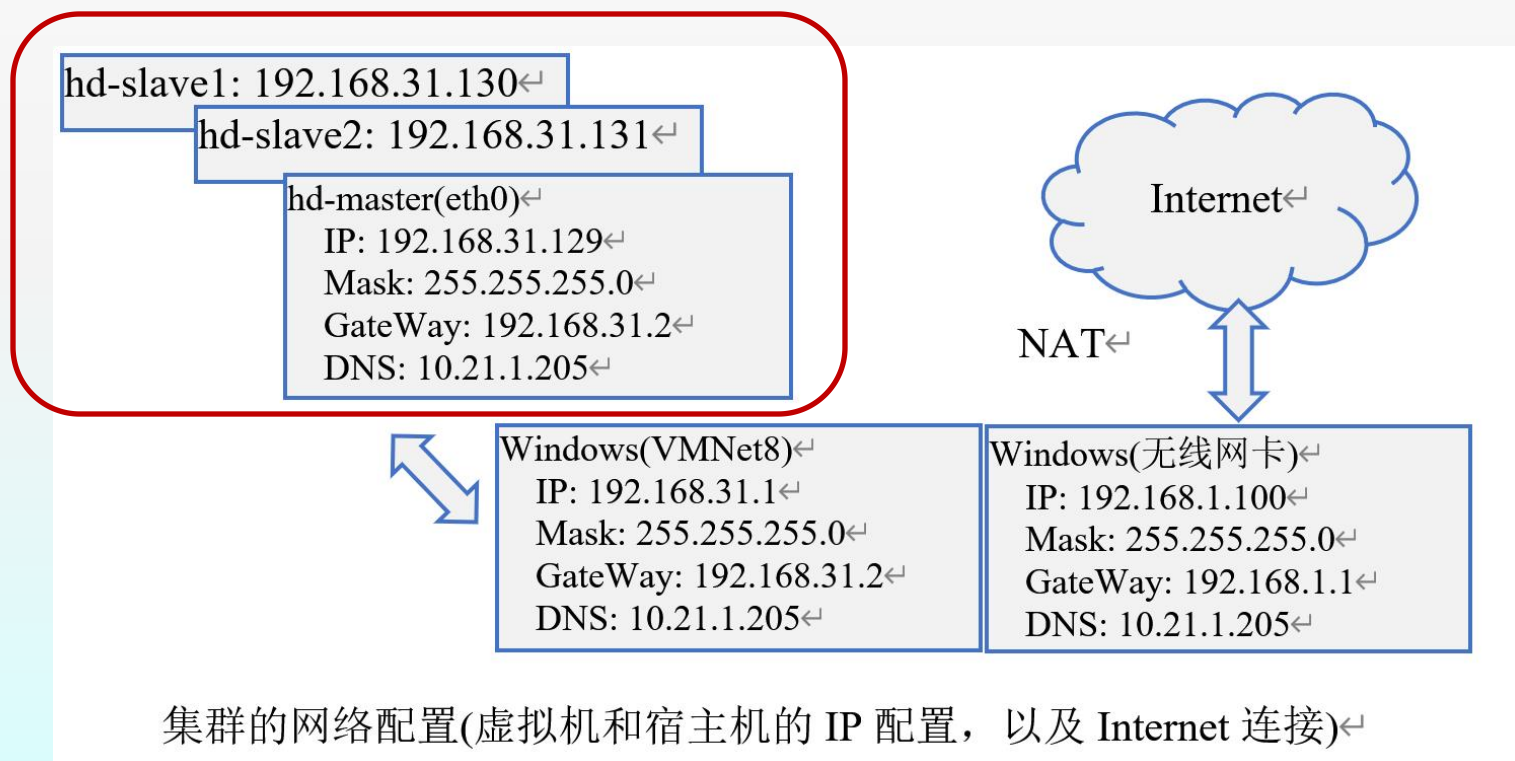
- ◆ 在一台虚拟机上安装了CentOS以后，可以复制该虚拟机(复制虚拟机Image所在目录)，并且修改主机名、进行网络配置修改(IP地址)，构造3台虚拟机构成的集群
- ◆ 如果通过复制虚拟机Image的方式，建立另外两个虚拟机碰到问题，不好解决
- ◆ 可以参考第二个PPT所描述的，新建虚拟机2和虚拟机3，全新安装CentOS和进行相关配置，包括网络配置、Samba、SSHD、Yum等



集群示意图

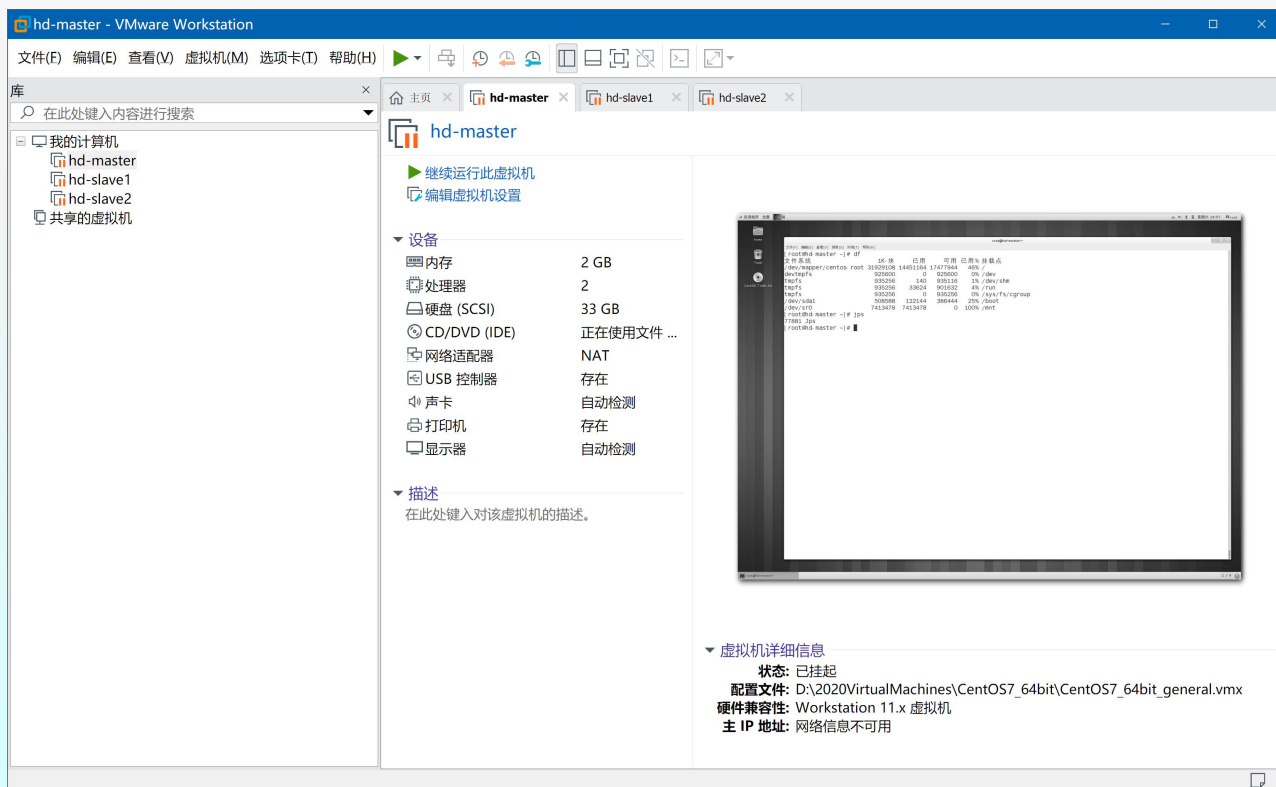
Hadoop安装与HDFS、MapReduce实验

- ◆ 新建虚拟机集群
- ◆ 集群网络配置



Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装与HDFS、MapReduce实验
 - ◆ 介绍Hadoop的安装和配置
 - ◆ 并且进行HDFS文件操作
 - ◆ MapReduce WordCount程序的实验



pause



Hadoop安装与HDFS、MapReduce实验

- ◆ **JDK**
- ◆ 为了安装Hadoop，首先需要安装特定版本的JDK
- ◆ 查看安装的JDK信息

```
rpm -qa | grep java
```

- ◆ **卸载当前的jdk**

```
rpm -e --nodeps java-1.7.0-openjdk-headless-1.7.0.75-2.5.4.2.el7_0.x86_64  
rpm -e --nodeps java-1.8.0-openjdk-headless-1.8.0.31-2.b13.el7.x86_64  
  
rpm -e --nodeps tzdata-java-2015a-1.el7.noarch  
  
rpm -e --nodeps java-1.8.0-openjdk-1.8.0.31-2.b13.el7.x86_64  
rpm -e --nodeps java-1.7.0-openjdk-1.7.0.75-2.5.4.2.el7_0.x86_64  
rpm -e --nodeps java-1.6.0-openjdk-1.6.0.34-1.13.6.1.el7_0.x86_64
```


Hadoop安装与HDFS、MapReduce实验

- ◆ JDK
 - ◆ 下载jdk-8u144-linux-x64.tar.gz
 - ◆ 解压和安装

```
mkdir -p /opt/linuxsir/java
cd /opt/linuxsir/java

tar -zxvf /opt/linuxsir/jdk-8u144-linux-x64.tar.gz
ls jdk1.8.0_144
mv jdk1.8.0_144 jdk

ls /opt/linuxsir/java/jdk
```

Hadoop安装与HDFS、MapReduce实验

- ◆ JDK
 - ◆ 配置JDK
 - ◆ 编辑/root/.bashrc文件，增加如下内容

```
ls /opt/linuxsir/java/jdk/jre/lib/rt.jar      \\查看文件是否存在
ls /opt/linuxsir/java/jdk/lib/dt.jar
ls /opt/linuxsir/java/jdk/lib/tools.jar

echo "export JAVA_HOME=/opt/linuxsir/java/jdk" >> /root/.bashrc
echo "export JRE_HOME=\$JAVA_HOME/jre" >> /root/.bashrc
echo "export PATH=\$JAVA_HOME/bin:\$JRE_HOME/bin:\$PATH" >>
/root/.bashrc

echo "export
CLASSPATH=.:\$CLASSPATH:\$JAVA_HOME/jre/lib/rt.jar:\$JAVA_HOME/lib/dt.j
ar:\$JAVA_HOME/lib/tools.jar" >> /root/.bashrc

cat /root/.bashrc
```

```
cd          \\进入/root目录，即root用户的主目录
source .bashrc
```


Hadoop安装与HDFS、MapReduce实验

- ◆ JDK
- ◆ 同步三台虚拟机

```
scp -r /opt/linuxsir/java/jdk root@192.168.31.130:/opt/linuxsir/java \\拷贝jdk
```

```
scp -r /opt/linuxsir/java/jdk root@192.168.31.131:/opt/linuxsir/java
```

```
scp -r /root/.bashrc root@192.168.31.130:/root/.bashrc \\拷贝/root/.bashrc
```

```
scp -r /root/.bashrc root@192.168.31.131:/root/.bashrc
```

```
ssh root@192.168.31.130 source ~/.bashrc \\刷新环境变量
```

```
ssh root@192.168.31.131 source ~/.bashrc
```

```
ssh root@192.168.31.130 java -version \\查看JDK版本
```

```
ssh root@192.168.31.131 java -version
```

Hadoop安装与HDFS、MapReduce实验

- ◆ 配置各个虚拟机别名/**etc/hostname**
 - ◆ 为192.168.31.129、192.168.31.130、192.168.31.131等3台虚拟机配置别名
 - ◆ 三台机器的/etc/hostname的内容分别是hd-master、hd-slave1、hd-slave2

在192.168.31.129上

```
cat /etc/hostname
```

```
rm -rf /etc/hostname
```

```
touch /etc/hostname
```

```
echo "hd-master" >> /etc/hostname
```

```
cat /etc/hostname| grep hd-master
```

- ◆ 另外2个主机运行类似的命令，注意主机名分别为hd-slave1、hd-slave2

Hadoop安装与HDFS、MapReduce实验

- ◆ 配置各个虚拟机的/etc/hosts文件
 - ◆ 192.168.31.129、192.168.31.130、192.168.31.131三台虚拟机的/etc/hosts内容一样

```
192.168.31.129 hd-master  
192.168.31.130 hd-slave1  
192.168.31.131 hd-slave2
```

```
127.0.0.1 localhost          \\ 这行放在最后
```

Hadoop安装与HDFS、MapReduce实验

- ◆ 配置各个虚拟机的/etc/hosts文件
- ◆ 192.168.31.129、192.168.31.130、192.168.31.131三台虚拟机的/etc/**hosts**内容一样

```
192.168.31.129 hd-master  
192.168.31.130 hd-slave1  
192.168.31.131 hd-slave2
```

```
127.0.0.1 localhost
```

\\ 这行放在最后

在三台虚拟机上执行如下命令

```
rm -rf /etc/hosts
```

```
touch /etc/hosts
```

```
echo "192.168.31.129 hd-master" >>/etc/hosts
```

```
echo "192.168.31.130 hd-slave1" >>/etc/hosts
```

```
echo "192.168.31.131 hd-slave2" >>/etc/hosts
```

```
echo "127.0.0.1 localhost" >>/etc/hosts
```

```
echo "" >>/etc/hosts
```

```
cat /etc/hosts
```

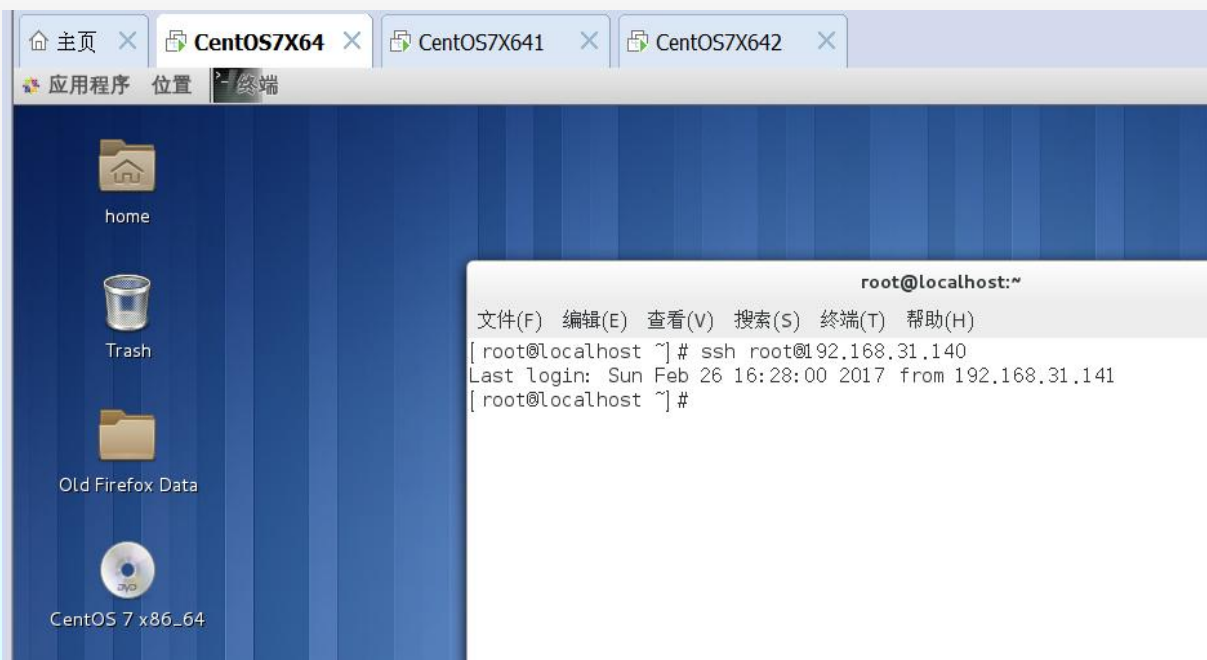
pause



Hadoop安装与HDFS、MapReduce实验

◆ 无密码ssh登录

- ◆ Linux虚拟机之间的无密码ssh登录，使得我们可以在hd-master上，方便地启动Hadoop服务进程，这些服务进程包括主节点的进程和从节点的进程
- ◆ 当从主节点启动从节点的进程的时候，系统无需等待用户输入密码



Hadoop安装与HDFS、MapReduce实验

◆ 无密码ssh登录

- ◆ 在192.168.31.129、192.168.31.130、192.168.31.131三台虚拟机上执行如下命令，配置sshd
- ◆ 编辑/etc/ssh/sshd_config，去掉以下两行注释，即把行首的#去掉，并且设置AuthorizedKeysFile

```
#RSAAuthentication yes    # 启用 RSA 认证
#PubkeyAuthentication yes  # 启用公钥私钥配对认证方式

AuthorizedKeysFile .ssh/authorized_keys    # 公钥文件路径(和下面生成的文件同名)
```


Hadoop安装与HDFS、MapReduce实验

- ◆ 无密码ssh登录
 - ◆ 在192.168.31.129、192.168.31.130、192.168.31.131上执行如下命令，生成key

```
whoami  
cd  
pwd
```

```
ssh-keygen -t rsa  
ls /root/.ssh/id_rsa
```

```
\\生成密钥文件  
\\文件在/root/.ssh/id_rsa
```

Hadoop安装与HDFS、MapReduce实验

- ◆ 无密码ssh登录

- ◆ 在192.168.1.129上执行如下命令，从另外两台机器(130/131)，合并公钥到authorized_keys文件

```
cd ~/.ssh  
ls authorized_keys  
rm -rf authorized_keys
```

```
cat id_rsa.pub >> authorized_keys  
ssh root@192.168.31.130 cat ~/.ssh/id_rsa.pub >> authorized_keys  
ssh root@192.168.31.131 cat ~/.ssh/id_rsa.pub >> authorized_keys
```

Hadoop安装与HDFS、MapReduce实验

- ◆ 无密码ssh登录
- ◆ 从192.168.31.129, 复制 ~/.ssh/authorized_keys 和 ~/.ssh/known_hosts 两个文件到另外两台机器192.168.31.130、192.3168.1.131
- ◆ 这时候, 每台机器(CentOS)均有3台机器的公钥

```
cd ~/.ssh
scp authorized_keys root@192.168.31.130:~/.ssh
scp authorized_keys root@192.168.31.131:~/.ssh

scp known_hosts root@192.168.31.130:~/.ssh
scp known_hosts root@192.168.31.131:~/.ssh
```

Hadoop安装与HDFS、MapReduce实验

◆ 无密码ssh登录

- ◆ 在192.168.31.129上执行如下命令，修改192.168.31.129/130/131等三台机器上的 ~/.ssh/authorized_keys 权限

```
cd ~/.ssh
ls authorized_keys
ssh root@192.168.31.130 ls ~/.ssh/authorized_keys
ssh root@192.168.31.131 ls ~/.ssh/authorized_keys

cd ~/.ssh
chmod 700 authorized_keys
ssh root@192.168.31.130 chmod 700 ~/.ssh/authorized_keys
ssh root@192.168.31.131 chmod 700 ~/.ssh/authorized_keys
```

Hadoop安装与HDFS、MapReduce实验

- ◆ 无密码ssh登录
 - ◆ 在192.168.31.129、192.168.31.130、192.3168.1.131上执行如下命令，重启sshd

```
service sshd restart  
\\或者用/etc/init.d/sshd restart
```

Hadoop安装与HDFS、MapReduce实验

- ◆ 无密码ssh登录
 - ◆ 测试无密码登录

\\在192.168.31.129上，测试无密码登录130/131

```
ssh root@192.168.31.130
```

```
exit
```

```
ssh root@192.168.31.131
```

```
exit
```

\\在192.168.31.130上，测试无密码登录129/131

```
ssh root@192.168.31.129
```

```
exit
```

```
ssh root@192.168.31.131
```

```
exit
```

\\在192.168.31.131上，测试无密码登录129/130

```
ssh root@192.168.31.129
```

```
exit
```

```
ssh root@192.168.31.130
```

```
exit
```

pause



Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 下载Hadoop，我们使用的版本是Hadoop2.7.3
 - ◆ 解压缩Hadoop安装包，并且对安装目录进行重命名

```
cd /opt/linuxsir  
tar -zxvf hadoop-2.7.3.tar.gz  
  
ls  
mv hadoop-2.7.3 /opt/linuxsir/hadoop
```

Hadoop安装与HDFS、MapReduce实验

◆ Hadoop安装、配置和启动

- ◆ 在192.168.31.129虚拟机上编辑/root/.bashrc文件，然后复制到192.168.31.130、192.168.31.131

```
echo "" >> /root/.bashrc
echo "export HADOOP_PREFIX=/opt/linuxsir/hadoop" >> /root/.bashrc
echo "export HADOOP_HOME=\$HADOOP_PREFIX" >> /root/.bashrc
echo "export HADOOP_COMMON_HOME=\$HADOOP_PREFIX" >> /root/.bashrc
echo "export HADOOP_CONF_DIR=\$HADOOP_PREFIX/etc/hadoop" >> /root/.bashrc
echo "export HADOOP_HDFS_HOME=\$HADOOP_PREFIX" >> /root/.bashrc
echo "export HADOOP_MAPRED_HOME=\$HADOOP_PREFIX" >> /root/.bashrc
echo "export HADOOP_YARN_HOME=\$HADOOP_PREFIX" >> /root/.bashrc
echo "export PATH=\$PATH:\$HADOOP_PREFIX/sbin:\$HADOOP_PREFIX/bin" >> /root/.bashrc

echo "export HADOOP_OPTS=\"-Djava.library.path=\$HADOOP_HOME/lib/native\"" >>
/root/.bashrc
echo "export HADOOP_COMMON_LIB_NATIVE_DIR=\$HADOOP_HOME/lib/native" >>
/root/.bashrc

echo "export CLASSPATH=\$CLASSPATH:/opt/linuxsir/hadoop/lib/*" >> /root/.bashrc

cat /root/.bashrc
```

Hadoop安装与HDFS、MapReduce实验

◆ Hadoop安装、配置和启动

- ◆ 在192.168.31.129虚拟机上编辑/root/.bashrc文件，然后复制到192.168.31.130、192.168.31.131

\\在192.168.31.129上，复制/root/.bashrc到130和131

```
scp -r /root/.bashrc root@192.168.31.130:/root/.bashrc
```

```
scp -r /root/.bashrc root@192.168.31.131:/root/.bashrc
```

\\在192.168.31.129上，在129/130/131三台机器上运行/root/.bashrc，刷新环境
cd

```
source /root/.bashrc
```

```
ssh root@192.168.31.130 source /root/.bashrc
```

```
ssh root@192.168.31.131 source /root/.bashrc
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
- ◆ 创建数据存放目录

```
cd /opt/linuxsir/hadoop          \\进入/opt/linuxsir/hadoop目录
rm -rf /opt/linuxsir/hadoop/tmp
rm -rf /opt/linuxsir/hadoop/hdfs
mkdir /opt/linuxsir/hadoop/tmp    \\创建tmp目录
mkdir -p /opt/linuxsir/hadoop/hdfs/data /opt/linuxsir/hadoop/hdfs/name \\创建
hdfs的data、name子目录
```

\\还要针对hd-slave1,hd-slave2等两个节点上执行上述命令，然后再初始化hdfs

```
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/hdfs
ssh root@192.168.31.130 mkdir /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.130 mkdir -p /opt/linuxsir/hadoop/hdfs/data
/opt/linuxsir/hadoop/hdfs/name
```

```
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/hdfs
ssh root@192.168.31.131 mkdir /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.131 mkdir -p /opt/linuxsir/hadoop/hdfs/data
/opt/linuxsir/hadoop/hdfs/name
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 对若干配置文件进行设置，保证Hadoop能够正常启动。
 - ◆ (1) 主要的配置文件包括HADOOP_HOME目录下的
 - ◆ etc/hadoop/core-site.xml
 - ◆ etc/hadoop/hdfs-site.xml
 - ◆ etc/hadoop/yarn-site.xml
 - ◆ etc/hadoop/mapred-site.xml等文件
 - ◆ (2) 并且为如下文件配置环境变量
 - ◆ etc/hadoop/hadoop-env.sh
 - ◆ etc/hadoop/yarn-env.sh
 - ◆ etc/hadoop/mapred-env.sh等
 - ◆ (3) master和slave
 - ◆ masters
 - ◆ slaves

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 编辑/opt/linuxsir/hadoop/etc/hadoop目录下的core-site.xml文件

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>file:///opt/linuxsir/hadoop/tmp</value>
  </property>

  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://hd-master:9000</value><!-- NameNode URI -->
  </property>

  <property>
    <name>io.file.buffer.size</name>
    <value>131702</value>
  </property>
</configuration>
```

Hadoop安装与HDFS、MapReduce实验

◆ Hadoop安装、配置和启动

- ◆ 编辑/opt/linuxsir/hadoop/etc/hadoop目录下的hdfs-site.xml文件，内容如下

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///opt/linuxsir/hadoop/hdfs/name</value> <!-- 本机name目录for NameNode -->
  </property>

  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///opt/linuxsir/hadoop/hdfs/data</value> <!-- 本机data目录for DataNode -->
  </property>

  <property>
    <name>dfs.replication</name> <!-- 数据块副本数量 -->
    <value>2</value>
  </property>

  <property>
    <name>dfs.webhdfs.enabled</name>
    <value>true</value>
  </property>

  <property>
    <name>dfs.namenode.secondary.http-address</name>
    <value>hd-master:9001</value>
  </property>
</configuration>
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 在/opt/linuxsir/hadoop/etc/hadoop目录下，复制mapred-site.xml.template到mapred-site.xml，并且进行编辑

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
<property>
  <name>mapreduce.framework.name</name>
  <value>yarn</value> <!--yarn or yarn-tez-->
</property>

<property>
  <name>mapreduce.jobhistory.address</name>
  <value>hd-master:10020</value>
</property>

<property>
  <name>mapreduce.jobhistory.webapp.address</name>
  <value>hd-master:19888</value>
</property>
```


Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动

- ◆ 在/opt/linuxsir/hadoop/etc/hadoop目录下，复制mapred-site.xml.template到mapred-site.xml，并且进行编辑

```
<property>
  <name>mapreduce.map.memory.mb</name> <!-- memory for map task -->
  <value>64</value>
</property>
<property>
  <name>mapreduce.reduce.memory.mb</name> <!-- memory for reduce task -->
  <value>128</value>
</property>
<property>
  <name>mapreduce.task.io.sort.mb</name>
  <value>32</value>
</property>

<property>
  <name>mapreduce.map.java.opts</name> <!-- settings for JVM map task -->
  <value>-Xms128m -Xmx256m</value>
</property>
<property>
  <name>mapreduce.reduce.java.opts</name> <!-- settings for JVM reduce task -->
  <value>-Xms128m -Xmx256m</value>
</property>
</configuration>
```

Hadoop安装与HDFS、MapReduce实验

◆ Hadoop安装、配置和启动

- ◆ 在/opt/linuxsir/hadoop/etc/hadoop编辑yarn-site.xml文件，对YARN资源管理器的ResourceManager和NodeManagers节点、端口、内存分配等进行配置

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
<property>
  <name>yarn.resourcemanager.hostname</name>
  <value>hd-master</value>
</property>

<property>
  <name>yarn.resourcemanager.address</name>
  <value>hd-master:9032</value>
</property>
<property>
  <name>yarn.resourcemanager.scheduler.address</name>
  <value>hd-master:9030</value>
</property>
<property>
  <name>yarn.resourcemanager.resource-tracker.address</name>
  <value>hd-master:9031</value>
</property>
<property>
  <name>yarn.resourcemanager.admin.address</name>
  <value>hd-master:9033</value>
</property>
<property>
  <name>yarn.resourcemanager.webapp.address</name>
  <value>hd-master:9099</value>
</property>
```

Hadoop安装与HDFS、MapReduce实验

◆ Hadoop安装、配置和启动

- ◆ 在/opt/linuxsir/hadoop/etc/hadoop编辑yarn-site.xml文件，对YARN资源管理器的ResourceManager和NodeManagers节点、端口、内存分配等进行配置

```
<property>
  <name>yarn.nodemanager.resource.memory-mb</name>
  <value>2048</value>
</property>
<property>
  <name>yarn.scheduler.maximum-allocation-mb</name>
  <value>2048</value>
</property>
<property>
  <name>yarn.scheduler.minimum-allocation-mb</name>
  <value>1024</value>
</property>
<property>
  <name>yarn.app.mapreduce.am.resource.mb</name>
  <value>1024</value>
</property>
<property>
  <name>yarn.app.mapreduce.am.command-opts</name>
  <value>-Xms128m -Xmx256m</value>
</property>
```

Hadoop安装与HDFS、MapReduce实验

◆ Hadoop安装、配置和启动

- ◆ 在/opt/linuxsir/hadoop/etc/hadoop编辑yarn-site.xml文件，对YARN资源管理器的ResourceManager和NodeManagers节点、端口、内存分配等进行配置

```
<property>
  <name>yarn.nodemanager.vmem-check-enabled</name>
  <value>>false</value>
</property>
<property>
  <name>yarn.nodemanager.vmem-pmem-ratio</name>
  <value>8</value>
</property>
<property>
  <name>yarn.nodemanager.resource.cpu-vcores</name>
  <value>1</value>
</property>

<property>
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
</property>
<property>
  <name>yarn.nodemanager.aux-services.mapreduce.shuffle.class</name>
  <value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
</configuration>
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 进入 `hadoop-env.sh` 脚本文件所在目录
`/opt/linuxsir/hadoop/etc/Hadoop`

```
export JAVA_HOME=/opt/linuxsir/java/jdk
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 接着，设置/opt/linuxsir/hadoop/etc/hadoop目录下yarn-env.sh脚本文件的JAVA_HOME变量，内容如下

```
export JAVA_HOME=/opt/linuxsir/java/jdk
```

- ◆ 如果NodeManager因为内存不足，而启动不起来，那么yarn-env.sh文件需要做如下修改，即JAVA_HEAP_MAX改为3G

```
JAVA_HEAP_MAX=-Xmx3072m
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 修改/opt/linuxsir/hadoop/etc/hadoop/**masters**文件和/opt/linuxsir/hadoop/etc/hadoop/**slaves**文件，目的是指定主节点和从节点列表。
 - ◆ /opt/linuxsir/hadoop/etc/hadoop/**masters**文件的内容如下，即主节点为hd-master

```
hd-master
```

- ◆ /opt/linuxsir/hadoop/etc/hadoop/**slaves**文件的内容如下，即从节点为hd-slave1和hd-slave2

```
hd-slave1  
hd-slave2
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 从192.168.31.129虚拟机复制Hadoop到其它各个节点，包括192.168.31.130、192.168.31.131。
 - ◆ 在192.168.31.129上运行如下命令

```
chmod a+rwx -R /opt/linuxsir \\设置/opt/linuxsir的存取权限  
ssh root@192.168.31.130 chmod a+rwx -R /opt/linuxsir  
ssh root@192.168.31.131 chmod a+rwx -R /opt/linuxsir
```

```
scp -r /root/.bashrc root@192.168.31.130:/root/.bashrc \\复制/root/.bashrc  
scp -r /root/.bashrc root@192.168.31.131:/root/.bashrc
```

```
scp -r /opt/linuxsir/hadoop hd-slave1:/opt/linuxsir \\复制/opt/linuxsir/hadoop  
scp -r /opt/linuxsir/hadoop hd-slave2:/opt/linuxsir
```

```
source ~/.bashrc \\刷新环境变量  
ssh root@192.168.31.130 source ~/.bashrc  
ssh root@192.168.31.131 source ~/.bashrc
```


pause



Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 格式化HDFS
 - ◆ 启用HDFS文件系统之前，需要对其进行格式化；格式化只需做一次
 - ◆ 在192.168.31.129上执行如下命令

```
cd /opt/linuxsir/hadoop/bin  
./hdfs namenode -format
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 启动Hadoop
 - ◆ 清理hadoop日志

```
rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 启动Hadoop

```
cd /opt/linuxsir/hadoop/sbin  
./start-all.sh
```

```
\\如果要停止，请执行如下命令  
cd /opt/linuxsir/hadoop/sbin  
./stop-all.sh
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 也可以分开启动HDFS和YARN

```
clear  
cd /opt/linuxsir/hadoop/sbin  
./start-dfs.sh  
./start-yarn.sh
```

```
\\如果要停止，请执行如下命令，即分开停  
止HDFS和YARN  
cd /opt/linuxsir/hadoop/sbin  
./stop-yarn.sh  
./stop-dfs.sh
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 现在，可以在三个节点上，查看进程，验证Hadoop是否成功启动

```
clear  
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

hd-master 的输出如下(进程号可能不一样)↵

3930 ResourceManager↵

4506 Jps↵

3693 NameNode↵

3695 SecondaryNameNode↵

↵

hd-slave1,hd-slave2 的输出如下(进程号可能不一样)↵

2792 NodeManager↵

2920 Jps↵

2701 DataNode↵

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 到目前为止，启动HDFS和YARN以后，各个节点的进程，如下图所示

yarn 层↵	ResourceManager↵	NodeManager↵	NodeManager↵	...↵	↵
hdfs 层↵	NameNode↵ Secondary NameNode↵	DataNode↵	DataNode↵	...↵	↵
Hardware 各个节点↵	hd-master 节点↵ 192.168.31.129↵	hd-slave1 节点↵ 192.168.31.130↵	hd-slave2 节点↵ 192.168.31.131↵	...↵	↵

各个节点的进程↵

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 在hd-master上运行如下命令，报告HDFS的基本信息

```
cd /opt/linuxsir/hadoop  
./bin/hdfs dfsadmin -report
```


Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动

- ◆ 使用日志

- ◆ 如果Hadoop启动出问题，可以通过查看日志来寻找原因。每次启动Hadoop，应该首先清空三个节点的logs目录，方便寻找错误。

- ◆ 当启动出错，可以到相应节点上，查看日志文件。**哪个节点启动出错，就看哪个节点的日志文件**。由于有无密码ssh登录，可以通过主节点登录到其它节点，去查看所有节点的日志文件。

- ◆ 日志文件分别在hd-master、hd-slave1、hd-slave2的**/opt/linuxsir/hadoop/logs**目录下。

- ◆ 启动Hadoop之前，删除log文件

- ◆ 如果启动出问题，log文件里就是最新的出错信息

```
rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 管理界面
 - ◆ 若干web管理界面，列表如下

访问NameNode管理页面，监控文件系统。

<http://192.168.31.129:50070/>

访问ResourceManager(整个Cluster)管理页面，监控集群状况。

<http://192.168.31.129:9099/>

\\这个端口缺省是8088，由于端口冲突，改成9099, 参考yarn-site.xml

MapReduce JobHistory Server的管理页面，查看MapReduce作业提交历史；需要事先启动JobHistory Server。

<http://192.168.31.129:19888/>

pause



Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS 常用文件操作命令

```
cd /opt/linuxsir/hadoop/bin
hdfs dfsadmin -safemode leave
\\ 用户可以通过dfsadmin -safemode value 来操作安全模式，参数value的说明如下：
\\ enter - 进入安全模式
\\ leave - 强制NameNode离开安全模式
\\ get - 返回安全模式是否开启的信息
\\ wait - 等待，一直到安全模式结束

cd /opt/linuxsir/hadoop/bin
./hdfs dfs -rmr /input           \\ 递归式删除目录
./hdfs dfs -mkdir /input         \\ 创建目录
./hdfs dfs -chmod a+rwx /input   \\ 授权

./hdfs dfs -mkdir /output        \\ 创建目录
./hdfs dfs -copyFromLocal /opt/linuxsir/test.txt /input \\ 拷贝文件到HDFS
\\ 或者./hdfs dfs -put /opt/linuxsir/test.txt /input

./hdfs dfs -cat /input/test.txt | head   \\ 显示文件的头几行
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 执行Wordcount MapReduce程序
 - ◆ 注意，需要事先启动HDFS和YARN

```
cd /opt/linuxsir/hadoop/bin
./hdfs dfs -cat /input/test.txt

./hadoop jar
/opt/linuxsir/hadoop/share/hadoop/mapreduce/hadoop-
mapreduce-examples-2.7.3.jar wordcount /input/test.txt /output
./hdfs dfs -ls /output
./hdfs dfs -cat /output/part-r-00000
```

- ◆ 为了运行wordcount，必须保证hdfs分布式文件系统的/output不存在。如果存在可以把它删除，命令如下

```
cd /opt/linuxsir/hadoop/bin
./hdfs dfs -ls /output

./hdfs dfs -rm /output/*
./hdfs dfs -rmdir /output
```

pause



Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ **配置History Server**
 - ◆ (1) 在hd-master节点上，配置History Server
 - ◆ 在.../etc/hadoop/mapred-site.xml中配置以下内容

```
<property>
  <name>mapreduce.jobhistory.address</name>
  <value>hd-master:10020</value>
</property>
<property>
  <name>mapreduce.jobhistory.webapp.address</name>
  <value>hd-master:19888</value>
</property>
```


Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ **配置History Server**
 - ◆ (2) 把hd-master的新配置分发到所有节点即hd-slave1和hd-slave2。

```
clear
```

```
scp /opt/linuxsir/hadoop/etc/hadoop/mapred-site.xml hd-slave1:/opt/linuxsir/hadoop/etc/hadoop  
scp /opt/linuxsir/hadoop/etc/hadoop/mapred-site.xml hd-slave2:/opt/linuxsir/hadoop/etc/hadoop
```


Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 配置History Server
 - ◆ (3) 启动服务，在hd-master这台服务器上执行以下语句。
 - ◆ 注意，需要事先启动HDFS和YARN

```
cd /opt/linuxsir/hadoop/sbin  
mr-jobhistory-daemon.sh start historyserver
```

```
clear  
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 配置History Server
 - ◆ (4) 访问MapReduce JobHistory Server

<http://192.168.31.129:19888/>

pause



Hadoop安装与HDFS、MapReduce实验

◆ Hadoop安装、配置和启动

◆ 问题1

- ◆ DataNode启动不全，总共2个DataNode，只启动1个。解决办法如下，即需要重新格式化HDFS

删除3个节点上/opt/linuxsir/hadoop目录下的./tmp, ./hdfs/data, ./hdfs/name等目录的内容。重建这些目录，再格式化HDFS

```
rm -rf /opt/linuxsir/hadoop/tmp
rm -rf /opt/linuxsir/hadoop/hdfs
```

```
mkdir /opt/linuxsir/hadoop/tmp      \\创建tmp目录
mkdir -p /opt/linuxsir/hadoop/hdfs/data /opt/linuxsir/hadoop/hdfs/name \\创建hdfs的data、name子目录
```

```
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/hdfs
```

```
ssh root@192.168.31.130 mkdir /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.130 mkdir -p /opt/linuxsir/hadoop/hdfs/data /opt/linuxsir/hadoop/hdfs/name
```

```
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/hdfs
```

```
ssh root@192.168.31.131 mkdir /opt/linuxsir/hadoop/tmp
ssh root@192.168.31.131 mkdir -p /opt/linuxsir/hadoop/hdfs/data /opt/linuxsir/hadoop/hdfs/name
```

```
cd /opt/linuxsir/hadoop/bin
./hdfs namenode -format
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动

- ◆ **问题2**

- ◆ Hadoop相关进程(Node Manager)不能正常启动，在hd-slave1、hd-slave2上的/opt/linuxsir/hadoop/logs目录下，查看日志文件，发现Node Manager不能绑定到8040。
 - ◆ 提示信息为“error starting nodemanager does not contain a valid host:port authority 8040.”

上述问题的解决办法是修改hd-master的etc/hadoop/**yarn-site.xml**配置文件，设定9030、9031、9032、9033等端口，可以参考前文给出的yarn-site.xml的内容

并且把修改传播到整个集群

```
scp -r /opt/linuxsir/hadoop/etc/hadoop/yarn-site.xml hd-slave1:/opt/linuxsir/hadoop/etc/hadoop
scp -r /opt/linuxsir/hadoop/etc/hadoop/yarn-site.xml hd-slave2:/opt/linuxsir/hadoop/etc/hadoop
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动

- ◆ **问题3**

- ◆ 提示信息 “Invalid resource request, requested memory < 0, or requested memory > max configured, requestedMemory=1536, maxMemory=1024.”

对配置文件`yarn-site.xml`做如下修改

```
<property>
  <name>yarn.nodemanager.resource.memory-mb</name>
  <value>2048</value>
</property>
<property>
  <name>yarn.scheduler.maximum-allocation-mb</name>
  <value>2048</value>
</property>
<property>
  <name>yarn.scheduler.minimum-allocation-mb</name>
  <value>1024</value>
</property>
<property>
  <name>yarn.app.mapreduce.am.resource.mb</name>
  <value>1024</value>
</property>
<property>
  <name>yarn.app.mapreduce.am.command-opts</name>
  <value>-Xms128m -Xmx256m</value>
</property>
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动

- ◆ **问题3**

- ◆ 提示信息 “Invalid resource request, requested memory < 0, or requested memory > max configured, requestedMemory=1536, maxMemory=1024.”

对配置文件`mapred-site.xml`做如下修改

```
<property>
  <name>mapreduce.map.java.opts</name><!-- settings for JVM map task -->
  <value>-Xms128m -Xmx256m</value>
</property>
<property>
  <name>mapreduce.reduce.java.opts</name><!-- settings for JVM reduce task -->
  <value>-Xms128m -Xmx256m</value>
</property>
```

并且把修改传播到整个集群

```
scp -r /opt/linuxsir/hadoop/etc/hadoop/yarn-site.xml hd-slave1:/opt/linuxsir/hadoop/etc/hadoop
scp -r /opt/linuxsir/hadoop/etc/hadoop/yarn-site.xml hd-slave2:/opt/linuxsir/hadoop/etc/hadoop
```

```
scp -r /opt/linuxsir/hadoop/etc/hadoop/mapred-site.xml hd-slave1:/opt/linuxsir/hadoop/etc/hadoop
scp -r /opt/linuxsir/hadoop/etc/hadoop/mapred-site.xml hd-slave2:/opt/linuxsir/hadoop/etc/hadoop
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ **问题4**
 - ◆ resource manager的8032端口被占用，连接不上

参考问题2解决办法

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动

- ◆ **问题5**

- ◆ 提示信息 “running beyond virtual memory limits. Current usage: 49.2 MB of 384 MB physical memory used; 1.6 GB of 806.4 MB virtual memory used. Killing container.” 。
 - ◆ 对配置文件yarn-site.xml做如下修改，表示把虚拟内存开辟到物理内存容量的8倍的大小

```
<property>
  <name>yarn.nodemanager.vmem-check-enabled</name>
  <value>>false</value>
</property>
<property>
  <name>yarn.nodemanager.vmem-pmem-ratio</name>
  <value>8</value>
</property>
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动

- ◆ **问题6**

- ◆ 提示信息 “running beyond physical memory limits. Current usage: 183.2 MB of 128 MB physical memory used; 817.1 MB of 1 GB virtual memory used. Killing container.”

参考问题3和问题5解决办法

pause



Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ 为了顺利运行该实例，需要编辑 `/opt/linuxsir/hadoop/etc/hadoop/hdfs-site.xml` 配置文件，添加如下配置

```
<!-- for windows access linux HDFS -->
<property>
  <name>dfs.permissions.enabled</name>
  <value>>false</value>
</property>
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ 启动HDFS和YARN

```
rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

```
clear  
cd /opt/linuxsir/hadoop/sbin  
./start-dfs.sh  
./start-yarn.sh
```

```
clear  
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ 存取HDFS的代码分析如下
 - ◆ 首先导入必要的Java类

```
package com.pai.hdfs_demo;

import org.apache.commons.io.IOUtils;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FSDataInputStream;
import org.apache.hadoop.fs.FSDataOutputStream;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;

import java.io.*;
import java.nio.charset.StandardCharsets;
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ **HDFS Java程序分析**
 - ◆ 新建一个类ReadWriteHDFSExample，编写main函数如下。main函数调用其它函数，创建目录，写入数据，添加数据，然后再读取数据

```
public class ReadWriteHDFSExample {  
  
    // main  
    public static void main(String[] args) throws IOException {  
        // ReadWriteHDFSExample.checkExists();  
  
        ReadWriteHDFSExample.createDirectory();  
        ReadWriteHDFSExample.writeFileToHDFS();  
        ReadWriteHDFSExample.appendToHDFSFile();  
  
        ReadWriteHDFSExample.readFileFromHDFS();  
    }  
}
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ **readFileFromHDFS**函数如下，该函数读取文件内容，以字符串形式显示出来

```
// readFileFromHDFS
public static void readFileFromHDFS() throws IOException {
    Configuration configuration = new Configuration();
    configuration.set("fs.defaultFS", "hdfs://192.168.31.129:9000");
    FileSystem fileSystem = FileSystem.get(configuration);

    // Create a path
    String fileName = "read_write_hdfs_example.txt";
    Path hdfsReadPath = new Path("/javareadwriteexample/" + fileName);
    // initialize input stream
    FSDataInputStream inputStream = fileSystem.open(hdfsReadPath);
    // Classical input stream usage
    String out = IOUtils.toString(inputStream, "UTF-8");
    System.out.println(out);

    // BufferedReader bufferedReader = new BufferedReader(
    // new InputStreamReader(inputStream, StandardCharsets.UTF_8));
    // String line = null;
    // while ((line=bufferedReader.readLine())!=null){
    // System.out.println(line);
    // }

    inputStream.close();
    fileSystem.close();
}
```


Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ **writeFileToHDFS**函数打开文件，写入一行文本

```
// writeFileToHDFS
public static void writeFileToHDFS() throws IOException {
    Configuration configuration = new Configuration();
    configuration.set("fs.defaultFS", "hdfs://192.168.31.129:9000");
    FileSystem fileSystem = FileSystem.get(configuration);

    // Create a path
    String fileName = "read_write_hdfs_example.txt";
    Path hdfsWritePath = new Path("/javareadwriteexample/" + fileName);
    FSDataOutputStream fsDataOutputStream = fileSystem.create(hdfsWritePath, true);

    BufferedWriter bufferedWriter = new BufferedWriter(
        new OutputStreamWriter(fsDataOutputStream, StandardCharsets.UTF_8));

    bufferedWriter.write("Java API to write data in HDFS");
    bufferedWriter.newLine();

    bufferedWriter.close();
    fileSystem.close();
}
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ **appendToHDFSFile**函数打开文件，添加一行文本。需要注意的是，需要对Configuration类的对象configuration进行适当设置，否则出错

```
// appendToHDFSFile
public static void appendToHDFSFile() throws IOException {
    Configuration configuration = new Configuration();
    configuration.set("fs.defaultFS", "hdfs://192.168.31.129:9000");
    //configuration.setBoolean("dfs.client.block.write.replace-datanode-on-failure.enabled", true);
    configuration.set("dfs.client.block.write.replace-datanode-on-failure.policy", "NEVER");
    configuration.set("dfs.client.block.write.replace-datanode-on-failure.enable", "true");
    FileSystem fileSystem = FileSystem.get(configuration);

    // Create a path
    String fileName = "read_write_hdfs_example.txt";
    Path hdfsWritePath = new Path("/javareadwriteexample/" + fileName);
    FSDataOutputStream fsDataOutputStream = fileSystem.append(hdfsWritePath);

    BufferedWriter bufferedWriter = new BufferedWriter(
        new OutputStreamWriter(fsDataOutputStream, StandardCharsets.UTF_8));

    bufferedWriter.write("Java API to append data in HDFS file");
    bufferedWriter.newLine();

    bufferedWriter.close();
    fileSystem.close();
}
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ **createDirectory**函数创建一个目录

```
// createDirectory
public static void createDirectory() throws IOException {
    Configuration configuration = new Configuration();
    configuration.set("fs.defaultFS", "hdfs://192.168.31.129:9000");
    FileSystem fileSystem = FileSystem.get(configuration);

    String directoryName = "/javareadwriteexample";
    Path path = new Path(directoryName);
    fileSystem.mkdirs(path);
}
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ **checkExists**检查目录或者文件是否存在。注意如下代码的最后一个括号是ReadWriteHDFSExample类的结束括号

```
// checkExists
public static void checkExists() throws IOException {
    Configuration configuration = new Configuration();
    configuration.set("fs.defaultFS", "hdfs://192.168.31.129:9000");
    FileSystem fileSystem = FileSystem.get(configuration);

    String directoryName = "/javareadwriteexample";
    Path path = new Path(directoryName);
    if (fileSystem.exists(path)) {
        System.out.println("File/Folder Exists : " + path.getName());
    } else {
        System.out.println("File/Folder does not Exists : " + path.getName());
    }
}
}
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ 为了编译通过上述Java代码，需要把如下目录下的jar包导入Eclipse项目的Build Path
 - ◆ 操作序列为“Eclipse Java项目→右键→Properties→Java Build Path→Libraries→Add External Jars”

D:\hadoop-2.7.3\share\hadoop\common\lib
D:\hadoop-2.7.3\share\hadoop\common
D:\hadoop-2.7.3\share\hadoop\hdfs

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ 执行完毕上述代码后，在hd-master主机上可以通过如下命令，**检查已经写入的文件**

```
cd /opt/linuxsir/hadoop/bin  
./hdfs dfs -ls /javareadwriteexample/read_write_hdfs_example.txt  
./hdfs dfs -cat /javareadwriteexample/read_write_hdfs_example.txt
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ HDFS Java程序分析
 - ◆ 为了多次进行实验(或者为了调试代码), 可以把HDFS文件删除, 然后再执行或者调试Java代码, 否则一经存在该目录, 执行创建目录的代码就会出错

```
cd /opt/linuxsir/hadoop/bin
./hdfs dfs -rm /javareadwriteexample/*
./hdfs dfs -rmdir /javareadwriteexample
```

- ◆ 停止YARN和HDFS

```
cd /opt/linuxsir/hadoop/sbin
./stop-yarn.sh
./stop-dfs.sh

jps
ssh root@192.168.31.130 jps
ssh root@192.168.31.131 jps
```

pause



Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ Word Count程序代码分析
 - ◆ (1) 首先导入必要的Java Package

```
package mywordcount;

import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ Word Count程序代码分析
 - ◆ (2) 定义WordCount类的内部类TokenizerMapper
 - ◆ 该类实现了map函数，把从文件读取的每个word变成一个形式为<word,1>的Key Value对，输出到map函数的参数context对象，由执行引擎完成Shuffle

```
public static class TokenizerMapper
    extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
        ) throws IOException, InterruptedException {
        StringTokenizer itr = new StringTokenizer(value.toString());
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ Word Count程序代码分析
 - ◆ (3) 定义WordCount类的内部类IntSumReducer
 - ◆ 该类实现了reduce函数，它收拢所有相同key的、形式为<word,1>的Key-Value对，对Value部分进行累加，输出一个计数

```
public static class IntSumReducer
    extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context
        ) throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
        String thekey = key.toString();
        int thevalue = sum;
    }
}
```

Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ Word Count程序代码分析
 - ◆ (4) 为WordCount类实现main函数
 - ◆ WordCount类的main函数，负责配置Job的若干关键的参数，并且启动这个Job。
 - ◆ 在main函数中，conf对象包含了一个属性即“fs.defaultFS”，它的值为“hdfs://192.168.31.129:9000”，使得WordCount程序知道如何存取HDFS

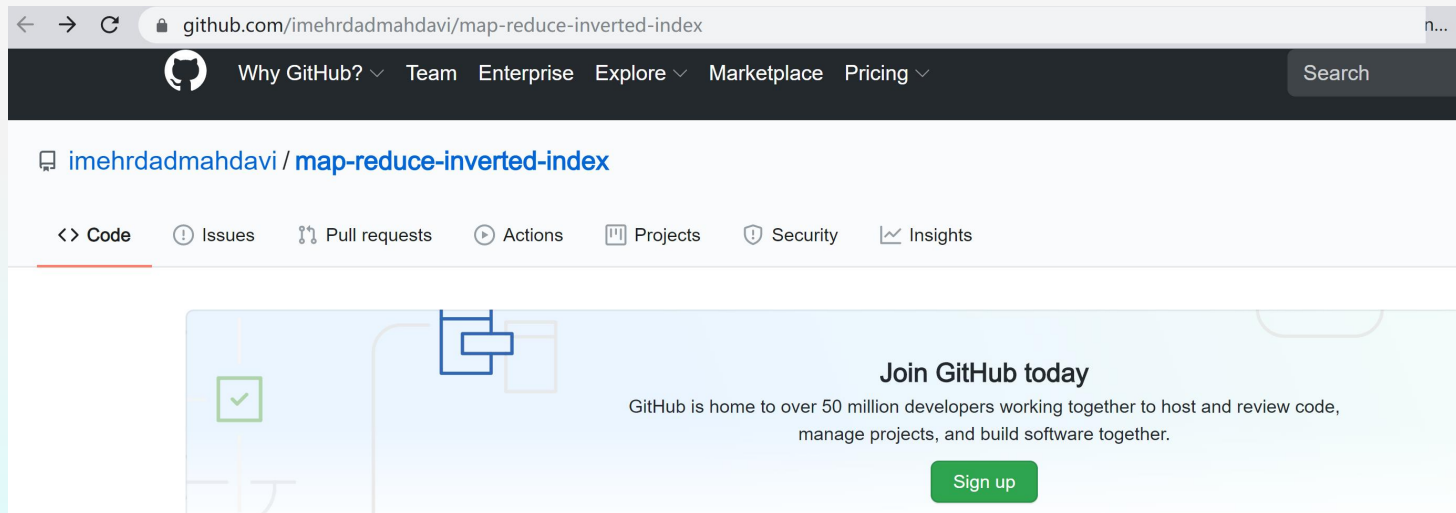
```
public class WordCount {  
  
    public static void main(String[] args) throws Exception {  
        Configuration conf = new Configuration();  
        String[] otherArgs = new GenericOptionsParser(conf, args).getRemainingArgs();  
        if (otherArgs.length != 2) {  
            System.err.println("Usage: wordcount <in> <out>");  
            System.exit(2);  
        }  
        conf.set("fs.defaultFS", "hdfs://192.168.31.129:9000");  
        Job job = new Job(conf, "word count");  
        job.setJarByClass(WordCount.class);  
        job.setMapperClass(TokenizerMapper.class);  
        job.setCombinerClass(IntSumReducer.class);  
        job.setReducerClass(IntSumReducer.class);  
        job.setOutputKeyClass(Text.class);  
        job.setOutputValueClass(IntWritable.class);  
        FileInputFormat.addInputPath(job, new Path(otherArgs[0]));  
        FileOutputFormat.setOutputPath(job, new Path(otherArgs[1]));  
  
        System.exit(job.waitForCompletion(true) ? 0 : 1);  
    }  
}
```

pause



Hadoop安装与HDFS、MapReduce实验

- ◆ Hadoop安装、配置和启动
 - ◆ 调研报告
 - ◆ 如何用Java编写程序建立倒排索引？



pause

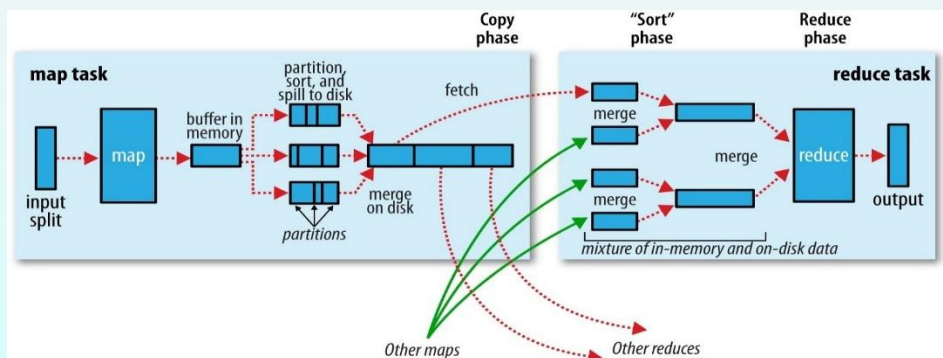


Hadoop安装与HDFS、MapReduce实验

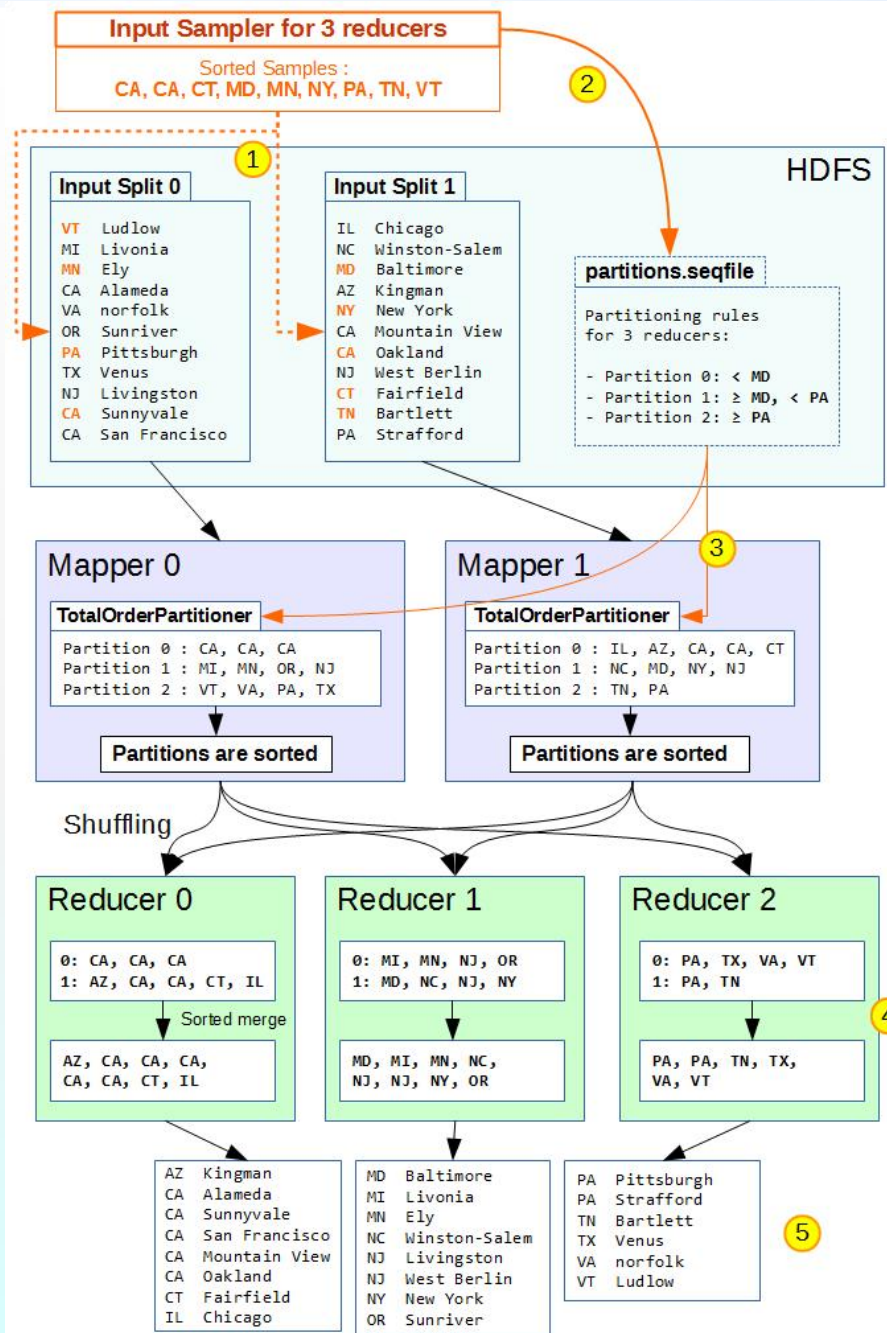
◆ Hadoop安装、配置和启动

◆ MapReduce Sort

- ◆ 当我们查看Terasort MapReduce应用程序的代码时，没有看到有关的排序功能。但是Terasort程序又能够对大规模的数据进行排序，这是为什么呢？这是由Mapreduce内部运行机制提供保证的。
- ◆ 每个mapper生成的数据首先缓存，缓存达到一定大小后，对数据进行排序，写入磁盘。一系列的磁盘小文件要进行合并，合并的过程，也保持数据的有序性。每个mapper的输出都划分为R份(Segment)，对应R个Reducer task。
- ◆ 对于每个reducer，从所有mappper接收应该由它处理的部分(Segment)，这个过程称为Shuffle。然后进行合并，合并的过程保持数据的有序性，即这是一个Merge Sort的过程



- 根据上述原理，我们看到Terasort程序的map函数和reduce函数都写得比较简单，Terasort程序依靠mapreduce运行平台的内部运行机制，实现了数据的排序



pause

